

COMPENDIO TÁCTICO: ALGORITMOS VORACES

Metodología, Justificación Teórica y Catálogo Total de Patrones

1. Estructura y Elementos del Esquema Voraz

Para cualquier ejercicio de teoría, debes identificar estos 5 elementos genéricos:

Elementos Fundamentales (Para definir en el examen)

- **Conjunto de candidatos (C):** Colección de elementos iniciales disponibles (objetos, monedas, tareas, aristas).
- **Solución factible (S):** Subconjunto de candidatos que cumple las restricciones y resuelve el problema.
- **Completable (Factibilidad):** Condición que asegura que añadir un candidato nuevo a la solución parcial no viola las reglas.
- **Función de selección:** Criterio para elegir el mejor candidato *en el momento actual* (ej: el de mayor valor).
- **Función objetivo:** Métrica final que se desea optimizar (ej: minimizar monedas, maximizar beneficio).

Esqueleto Universal de Pseudocódigo

```
Funcion Voraz(C: Conjunto) devuelve Solucion
  S = {}; // Inicializar solución factible vacía
  Ordenar(C, Función_Selección); // Aplicar el criterio voraz

  Mientras (C != {}) y (no Solución(S)):
    x = extraer_mejor(C); // Sacar el primero de la lista ordenada
    Si Completable(S U {x}) entonces:
      S = S U {x}; // Añadir a la solución definitiva
    Fin Si
  Fin Mientras

  Si Solución(S) devuelve S sino "No hay solución factible";
Fin Funcion
```

2. Metodología de Resolución: ¿Cómo afrontar el problema?

Sigue este protocolo mental y hazte estas preguntas para no fallar en el planteamiento:

- **PASO 1: Análisis del Problema (El Radar Voraz)**
 - ¿Qué me debo preguntar? → ¿Me piden maximizar o minimizar algo? ¿El problema se puede construir tomando decisiones de una en una de forma irrevocable?
 - *Acción:* Si la decisión parece irrevocable y pide un óptimo, asume que es Voraz.
- **PASO 2: Definición de los Candidatos y la Solución**
 - ¿Qué me debo preguntar? → ¿Qué estoy eligiendo en cada iteración? (Intervalos, vértices, pares).
 - *Acción:* Define C y S .
- **PASO 3: Diseño del Criterio de Selección (La Ordenación)**
 - ¿Qué me debo preguntar? → ¿Cuál es el recurso crítico? ¿Qué pasa si elijo el más pequeño, el más grande o el mejor ratio Beneficio/Coste?
 - *Acción:* Decide el criterio de ordenación (*sort*) inicial. Es el núcleo del algoritmo.
- **PASO 4: Definición de la Factibilidad (El If)**
 - ¿Qué me debo preguntar? → ¿Qué me prohíben estrictamente las reglas? (Ej: Solapar horarios, sobrepasar N kilos, crear ciclos en un grafo).
 - *Acción:* Escribe la condición exacta de la completabilidad.
- **PASO 5: Verificación Rápida (Romper el algoritmo)**
 - ¿Qué me debo preguntar? → ¿Puedo inventarme un caso de 3 elementos donde mi criterio tome una decisión estúpida?
 - *Acción:* Dibuja un mini-ejemplo. Si falla, revisa el criterio (suele arreglarse usando ratios).

3. Plantillas Mentales por Patrón (Catálogo de Tipologías)

- **A. Selección de Actividades / Tareas Solapadas:** *Objetivo:* Maximizar el número de tareas en 1 recurso. *Ordenación:* Por **Tiempo de Finalización** creciente. *Decisión:* La primera que quepa sin que su inicio pise el final de la última elegida.
- **B. Cobertura / Alcance (Gasolineras, antenas):** *Objetivo:* Minimizar paradas. *Ordenación:* Por **Posición geográfica** de origen a destino. *Decisión:* Desde donde estoy, busco la parada **más lejana posible** que siga siendo $\leq$ a mi autonomía máxima.
- **C. Two-Pointers / Emparejamiento (Esquíes, Galletas):** *Objetivo:* Emparejar minimizando diferencias. *Ordenación:* Ordenar **AMBOS** vectores creciente. *Decisión:* Si el objeto cumple, avanzar ambos. Si se queda corto, avanzar solo el objeto.
- **D. Fusión / Huffman (Árboles de Prefijos óptimos):** *Objetivo:* Minimizar el coste acumulado al unir elementos/frecuencias. *Estructura:* **Cola de Prioridad (Min-Heap)**. *Decisión:* Extraer los dos más pequeños, sumarlos, convertirlos en nodo padre y devolver la suma al Heap.
- **E. Mochila Fraccional (Polvo de oro, líquidos):** *Objetivo:* Maximizar valor sin pasarse de capacidad W , permitiendo partir objetos. *Ordenación:* Por **Ratio Beneficio / Peso** decreciente. *Decisión:* Meter enteros hasta que no quepan. El último objeto que no cabe entero, meter su fracción exacta.
- **F. Tareas con Plazos y Beneficios (Deadlines):** *Objetivo:* Maximizar beneficios haciendo tareas (1 u. de tiempo c/u) antes de su plazo límite d_i . *Ordenación:* Por **Beneficio descendente**. *Decisión:* Colocar la tarea lo **más tarde posible** sin superar su deadline. Si todos los huecos hacia atrás están llenos, descartar tarea.
- **G. Minimización del Tiempo de Espera (Sistema de colas):** *Objetivo:* Minimizar el tiempo medio que los clientes/procesos esperan en el sistema. *Ordenación:* Por **Tiempo de ejecución ascendente** (los más rápidos primero).
- **H. Grafos Clásicos (Kruskal, Prim y Dijkstra):**
 - *Kruskal (MST):* Ordenar aristas menor a mayor. Factibilidad: Que no formen ciclo (Union-Find).
 - *Prim (MST):* Crecer desde un nodo. Ordenar vértices conectados por menor coste (Min-Heap).
 - *Dijkstra (Caminos):* Recorrer red. Criterio: Menor distancia acumulada desde el origen.
- **I. Viajante de Comercio / TSP (Heurística Voraz):** *Objetivo:* Recorrido aproximado de coste mínimo pasando por todos los nodos. *Ordenación:* Aristas de menor a mayor. *Factibilidad:* 1) No superar grado 2 por vértice (no visitar tres veces). 2) No cerrar un ciclo a menos que tenga longitud V (todos los nodos visitados). *Nota:* No garantiza el óptimo, es una heurística.

4. Justificaciones de Optimalidad (Armamento Teórico)

Dependiendo del problema, un enfoque de demostración puede ser más sencillo que otro. Usa el que mejor encaje con la naturaleza de tu solución para no dejar cabos sueltos en el examen.

Opción 1: Argumento de Intercambio (La Demostración Universal)

Ideal para problemas de selección, emparejamiento o alcance. 1. Sea $A = \{a_1, \dots, a_k\}$ la solución construida por nuestro algoritmo voraz.

2. Supongamos que existe una solución óptima distinta O que es estrictamente mejor que A .
3. Al comparar ambas, sea x el primer elemento que nuestro algoritmo voraz eligió, pero que no está en O .
4. Dado nuestro **Criterio de Selección**, la solución O debió elegir en su lugar otro elemento y tal que y es "peor." igual.^a x bajo este criterio.
5. Si en O **intercambiamos** y por x , obtenemos una nueva solución O' que sigue siendo **factible** (no rompe reglas) y su Función Objetivo **no empeora**.
6. Repitiendo este intercambio, transformamos gradualmente O en A sin perder calidad. Contradice que O sea estrictamente mejor. **Nuestra solución voraz A es óptima.**

Opción 2: Lema de la Elección Segura (Greedy Choice Property)

Ideal para cuando quieres demostrar que "el primer paso voraz siempre es correcto". 1. Consideremos el elemento x , que es la elección voraz en el primer paso (ej. la tarea que termina antes).

2. Sea O una solución óptima cualquiera del problema.
3. Si $x \in O$, la elección voraz es parte de una solución óptima y hemos terminado.
4. Si $x \notin O$, sea y el elemento equivalente en O (ej. la primera tarea de O). Por la definición de nuestro criterio voraz, x es "mejor" que y (termina antes o tiene más valor).
5. Podemos sustituir y por x en O creando O' . Como x consume menos recursos o aporta más valor, O' sigue siendo factible y al menos tan buena como O .
6. Conclusión: Siempre existe una solución óptima que incluye nuestra decisión voraz.

Opción 3: Demostración por Cotas o Límites Matemáticos

Ideal cuando el problema se traduce en una ecuación matemática clara (ej. cortes de palabras en líneas). 1. Se establece matemáticamente cuál es el límite absoluto (la cota) para la función objetivo.
2. Se demuestra que cualquier solución factible está limitada por una variable V .
3. Se demuestra que nuestro algoritmo voraz alcanza exactamente el mejor valor posible de esa variable V (ej. ocupa todo el espacio sin desperdiciar nada).
4. Al alcanzar la cota teórica inmejorable, la solución debe ser, por definición, óptima.

Opción 4: Demostración por Teoría de Matroides (El escudo infalible)

El PDF fundamenta la optimalidad de muchos grafos en los Matroides. Un matroide es un par (S, I) donde S son los candidatos e I la familia de subconjuntos independientes (válidos). Para garantizar el óptimo, debes demostrar dos axiomas:

1. **Propiedad Hereditaria:** Si un conjunto de elementos A es válido ($A \in I$), cualquier subconjunto $B \subset A$ también debe serlo. (Ej: Si es factible hacer 3 tareas a tiempo, hacer solo 2 de ellas también lo es).
2. **Propiedad de Intercambio:** Si tienes dos conjuntos válidos A y B , siendo $|A| < |B|$, debe existir al menos un elemento x en $B \setminus A$, tal que al añadirlo a A ($A \cup \{x\}$) el conjunto siga siendo válido.

5. Análisis de Complejidades (Justificaciones Formales)

No pongas solo $\mathcal{O}(N \log N)$, desarrolla la explicación para obtener toda la puntuación.

- $\mathcal{O}(N \log N)$ - **Ordenación Dominante (El estándar)**
 - *Cuándo aplica:* Selección de actividades, Emparejamiento, Mochila Fraccional, Tareas con plazos.
 - *Justificación:* El algoritmo requiere ordenar el vector de N candidatos según el criterio voraz, lo cual cuesta $\mathcal{O}(N \log N)$. Luego, realiza un recorrido lineal con un bucle de coste $\mathcal{O}(N)$, comprobando la factibilidad en $\mathcal{O}(1)$. Por tanto, la ordenación domina asintóticamente: $\mathcal{O}(N \log N)$.
- $\mathcal{O}(N)$ - **Datos ya ordenados o Recorrido Simple**
 - *Cuándo aplica:* Variantes de gasolineras donde ya te dan el mapa ordenado por km, Sweep Line con datos ya procesados.
 - *Justificación:* Al proporcionar el enunciado los datos previamente ordenados, evitamos la ordenación. El algoritmo solo recorre la estructura de datos una única vez, evaluando la factibilidad en $\mathcal{O}(1)$. El coste total es $\mathcal{O}(N)$.
- $\mathcal{O}(N \log N)$ o $\mathcal{O}(A \log V)$ - **Uso de Cola de Prioridad (Min-Heaps)**
 - *Cuándo aplica:* Mezcla de ficheros, Algoritmo de Huffman, Dijkstra, Prim.
 - *Justificación:* Construir la Cola de Prioridad cuesta $\mathcal{O}(N)$. El bucle principal extrae elementos e inserta resultados $N - 1$ veces. Cada operación individual de reestructuración en el Heap cuesta $\mathcal{O}(\log N)$. Por tanto, efectuar N operaciones logarítmicas resulta en un coste total de $\mathcal{O}(N \log N)$.

6. Catálogo de Ejemplos de las Diapositivas de Teoría (El Mapeo Oficial)

Esta sección resume los problemas canónicos que se explican durante las clases teóricas. Son la base sobre la que se construyen todos los ejercicios de la asignatura. Se indica la ubicación exacta en el PDF de teoría (2-Voraces.pdf) para que puedas acudir a la fuente en segundos.

- **1. Problema del Cambio en Monedas** [Diapositivas 5 - 8]
Patrón: A (Variante de Selección con capacidad).
Decisión Voraz: Ordenar las monedas de mayor a menor valor. Elegir siempre la moneda más grande posible que no haga que la suma supere el importe a devolver.
Desarrollo conceptual: Coger monedas grandes reduce rápidamente el importe restante, minimizando el conteo total de monedas. La teoría demuestra que esto funciona para sistemas monetarios canónicos (1, 2, 5, 10, 20...), pero falla (no da el óptimo) si se introducen denominaciones arbitrarias (por ejemplo, si añades una moneda de 12 euros).
- **2. El Problema de la Mochila (Fraccional)** [Diapositivas 9 - 20]
Patrón: E (Mochila con fracciones).
Decisión Voraz: Ordenamos los objetos por su **rentabilidad** (ratio *valor/peso* en orden descendente). Llenamos la mochila con el objeto más rentable.
Desarrollo conceptual: Introducimos el 100 % de los objetos mientras quepan. En el momento crítico en que un objeto no cabe entero, tomamos exactamente la fracción que falta para llegar al límite de la mochila. Matemáticamente (diapositiva 16), esto maximiza el valor por cada gramo de capacidad consumida.

- **3. Códigos de Huffman (Compresión de datos)**

Patrón: D (Fusión en Min-Heap).

Decisión Voraz: Mantenemos los caracteres en una Cola de Prioridad ordenados por **frecuencia de aparición** (ascendente). Extraemos los dos de menor frecuencia, los fusionamos sumando sus frecuencias y volvemos a insertar el nodo padre.

Desarrollo conceptual: Al unir siempre los menos frecuentes en la parte inferior del árbol (y por ende, dotándoles de mayor profundidad), garantizamos que esos caracteres tendrán códigos binarios más largos. Esto reserva automáticamente los códigos binarios más cortos (cerca de la raíz) para los caracteres más comunes, comprimiendo el archivo final al máximo.

[Diapositivas 25 - 45]
- **4. Caminos Mínimos en Grafos (Algoritmo de Dijkstra)**

Patrón: H (Grafos - Dijkstra).

Decisión Voraz: Mantenemos los nodos descubiertos en una Cola de Prioridad ordenados por su **distancia acumulada desde el origen**. Extraemos el nodo más cercano no visitado y actualizamos las distancias de todos sus vecinos.

Desarrollo conceptual: Al consolidar como definitivo el camino más corto disponible en cada iteración, garantizamos que nunca encontraremos un "atajo" posterior que mejore esa métrica (la prueba matemática de esto asume obligatoriamente que no existen pesos negativos en el grafo).

[Diapositivas 54 - 66]
- **5. Árboles de Recubrimiento Mínimo (Kruskal y Prim)**

Patrón: H (Grafos).

Kruskal [Diap. 78]: Ordenar las **aristas** de menor a mayor peso. Iterar añadiéndolas al árbol final siempre y cuando no formen un ciclo (se controla eficientemente con una estructura de clases de equivalencia *Union-Find*).

Prim [Diap. 72]: Empezar la expansión desde cualquier **vértice**. Mantener en un Min-Heap los vértices adyacentes ordenados por el coste de conectarlos a la nube actual. Extraer el más barato.

[Diapositivas 67 - 77 (Prim) y 78 - 89 (Kruskal)]
- **6. Selección de Actividades (El Clásico Solapamiento)**

Patrón: A (Tareas solapadas).

Decisión Voraz: Ordenamos todas las actividades de menor a mayor según su **hora de finalización**. Iteramos tomando la primera actividad que sea compatible con el cierre de la anterior.

Desarrollo conceptual: Al elegir la actividad que termina antes, estamos liberando el recurso crítico lo más pronto posible, dejando el intervalo de tiempo futuro lo más grande posible para encajar otras actividades. La teoría advierte que elegir la tarea más corta o la que empieza antes es un error y no garantiza el óptimo.

[Diapositivas 72 - 77]
- **7. Minimización del Tiempo de Espera en el Sistema**

Patrón: G (Tiempos de espera).

Decisión Voraz: Ordenar a los clientes o procesos en orden **ascendente según su tiempo de ejecución**. Se atiende incondicionalmente primero al más rápido.

Desarrollo conceptual: El tiempo total de espera es la suma acumulada de los tiempos de todos los clientes. Si atendemos a una tarea inmensamente larga primero, ese gran bloque de tiempo penaliza sumándose a *todos* los clientes que esperan detrás. Atendiendo al rápido primero, su minúsculo tiempo penaliza muy poco al colectivo.

[Diapositivas 78 - 81]
- **8. Planificación de Tareas a Plazo Fijo (Deadlines)**

Patrón: F (Deadlines).

Decisión Voraz: Se ordena por **penalización (o beneficio) descendente**. La tarea con mayor penalización se intenta colocar en el calendario de tiempo **lo más cerca posible de su plazo límite** (buscando huecos hacia atrás). Si todos los huecos previos están ocupados, la tarea se descarta y se asume como "tardía".

Desarrollo conceptual: Priorizamos salvaguardar las tareas que nos harían perder más dinero si fallamos. Al empujarlas hacia el límite de su *deadline*, mantenemos libres los horarios tempranos para acomodar otras tareas urgentes. La corrección se fundamenta en la teoría de Matroides (Diapositiva 98).

[Diapositivas 93 - 99]
- **9. Heurísticas Voraces: Coloreado de Grafos**

Patrón: Heurística (No garantiza óptimo).

Decisión Voraz: Asignar un color inicial al primer nodo. Intentar propagar ese mismo color a todos los nodos posibles que no sean adyacentes a ninguno de los ya coloreados con ese tinte. Cuando el color no se pueda propagar más, coger un nuevo color y repetir el proceso.

Desarrollo conceptual: Esta estrategia falla (gasta más colores del mínimo necesario) si el grafo tiene un orden estructural malicioso de nodos. Sin embargo, dado que el problema del coloreado óptimo es de clase NP-Completo (tiempo exponencial), se acepta esta aproximación rápida en tiempo polinómico para usos prácticos.

[Diapositivas 103 - 106]
- **10. Heurísticas Voraces: Viajante de Comercio (TSP)**

Patrón: I (TSP Heurístico).

Decisión Voraz: Seleccionar la arista global más corta del mapa. Añadirla a la ruta si cumple dos reglas de factibilidad: 1) no hace que un nodo tenga grado mayor que 2 (visitarlo tres veces), y 2) no cierra un ciclo prematuro (antes de haber integrado todas las ciudades en la ruta).

Desarrollo conceptual: Este enfoque es puramente subóptimo (como demuestra la diapositiva 112, el resultado tiene un error del 3.3% respecto al óptimo real), pero es una solución de emergencia infinitamente más rápida que el tiempo factorial ($\mathcal{O}(N!)$) que exigiría calcular el ciclo exacto mediante Backtracking.

[Diapositivas 107 - 113]

7. Desarrollo de Decisiones Voraces (Ejercicios Clave de la Hoja de Prácticas)

Aquí desarrollamos analíticamente la lógica, el patrón exacto y la complejidad algorítmica de los ejercicios de tu lista para que puedas volcar la respuesta perfecta si aparece en el examen.

- **Ej. 2: Oferta de la Tienda BOGO (Compra 1, llévate 1 gratis $\leq$ al precio)**

Patrón: C (Adaptación de Two-Pointers sobre un solo array).

Coste: $\mathcal{O}(N \log N)$ (Dominado por la fase de ordenación inicial).

Desarrollo de la Decisión Voraz: Para maximizar el dinero ahorrado (los productos gratis), necesitamos que la tienda nos regale los ítems de mayor valor económico posibles. Como las reglas nos exigen que el regalo debe ser estrictamente menor o igual al ítem comprado, la estrategia óptima es emparejar artículos de precios muy similares en la franja más alta. Por tanto, **ordenamos todo el array de mayor a menor precio**. Iteramos saltando de 2 en 2: pagamos el artículo $A[i]$ y exigimos obligatoriamente que nos regalen el artículo $A[i + 1]$. Así aseguramos que la "pérdida" de valor entre lo que abonamos y lo que nos regalan sea la mínima matemáticamente posible.

- **Ej. 6: Cuadros y Puertas (Cobertura de puntos en una recta)**

Patrón: B (Cobertura / Alcance Geográfico).

Coste: $\mathcal{O}(N \log N)$ (Por requerir ordenar las coordenadas de las puertas).

Desarrollo de la Decisión Voraz: El objetivo es invertir el mínimo de cuadros. Primero, ordenamos el array de puertas por coordenadas geográficas de izquierda a derecha. Al escanear y encontrar la primera puerta sin cobertura, **no** centramos el cuadro geométricamente sobre ella. En lugar de eso, para maximizar el potencial de cobertura futura, colocamos el cuadro a la **máxima distancia legal hacia la derecha** que siga manteniendo a esa puerta dentro de su radio (es decir, lo situamos a $+0,75m$ de la puerta). Todo lo que quede bajo ese arco se marca como cubierto, y reiniciamos la lógica en la siguiente puerta huérfana.

- **Ej. 7: Mezcla de N Ficheros**

Patrón: D (Fusión Óptima - Algoritmo Puro de Huffman).

Coste: $\mathcal{O}(N \log N)$ (Por el mantenimiento del Min-Heap a lo largo de N fusiones).

Desarrollo de la Decisión Voraz: Si sumamos ficheros de forma lineal estricta ($A+B$, luego $+C$, luego $+D$), la inercia del tamaño del primer fichero penalizará sumándose repetitivamente al acumulado de todas las operaciones. La decisión radical y óptima es **insertar todos los tamaños independientes en una Cola de Prioridad (Min-Heap)**. Extraemos los dos menores del momento, efectuamos la suma, y volvemos a inyectar el archivo aglomerado resultante en el Heap. Al fusionar en todo momento los elementos más livianos del sistema, ahogamos la curva de crecimiento de los costes acumulativos de I/O dejándola en el mínimo absoluto.

- **Ej. 8 y 16: Gasolineras / Coche Eléctrico (Problema del Alcance)**

Patrón: B (Cobertura / Alcance Geográfico).

Coste: $\mathcal{O}(N)$ si nos entregan la ruta preordenada, $\mathcal{O}(N \log N)$ si exigen ordenarla.

Desarrollo de la Decisión Voraz: Para erradicar las paradas innecesarias, es vital exprimir cada fragmento de la batería al máximo. Evaluamos, partiendo de nuestra ubicación actual, el clúster de estaciones operativas que estén dentro de una distancia radial $\leq K$ (la autonomía residual). La directriz voraz nos obliga a **seleccionar incondicionalmente la estación más alejada posible dentro de dicho radio**, sobrepasando de largo todas las estaciones intermedias. Al atracar en el punto límite, efectuamos una recarga completa y ejecutamos un nuevo salto geográfico bajo el mismo prisma.

- **Ej. 11, 13 y 23: Esquíes y Galletas (Emparejamiento / Two-Pointers Puro)**

Patrón: C (Two-Pointers en estructuras disjuntas).

Coste: $\mathcal{O}(N \log N + M \log M)$ (La ordenación bivalente de ambos vectores domina la carga asintótica).

Desarrollo de la Decisión Voraz: Manejamos dos entidades de naturaleza asimétrica (ej. demanda infantil vs volumen de la galleta). Para prevenir la tragedia ineficiente de asignar un recurso de gran envergadura a un requerimiento mínimo, procedemos a **ordenar ambas matrices estructurales de menor a mayor**. Establecemos un iterador i en el requerimiento más leve y un iterador j en el recurso más pequeño. Efectuamos el cotejo: si la galleta j calma las exigencias del individuo i , ratificamos la asignación y adelantamos simultáneamente ambos punteros ($i++$, $j++$). Por el contrario, si la galleta ostenta un calibre demasiado ínfimo, sentenciamos que será inútil para cualquier niño venidero más ambicioso, procediendo a desecharla avanzando unívocamente el puntero del recurso ($j++$).

- **Ej. 20: Sumandos Compuestos (4, 6, 8, 9...)**

Patrón: Heurística Voraz Matemática.

Coste: $\mathcal{O}(1)$ (El problema se subsume a operaciones aritméticas modulares elementales).

Desarrollo de la Decisión Voraz: La meta de lograr el **mayor número combinatorio de sumandos** cuyo resultado de N impone lógicamente el uso masivo del menor dígito posible admisible. Puesto que el dominio exige números compuestos, nuestro pilar será el número 4. La resolución voraz dictamina agotar la base dividiendo el total $N/4$. El remanente de esta división euclídea ($N \pmod{4}$) nos pauta el parche final a aplicar:

- Si el resto es 0: Solución óptima estricta, la tupla constará netamente de cuatros.
- Si el resto es 1: Dado que no podemos apendar un '1', confiscamos uno de los cuatros cimentados ($4 + 1 = 5$, no válido) y un segundo cuatro ($4 + 4 = 8$, más 1 es 9). El ajuste voraz se basa en conmutar dos cuatros por

un 9.

- Si el resto es 2: Extirpamos un cuatro y le anexionamos el excedente ($4 + 2 = 6$, compuesto válido). El ajuste voraz es conmutar un 4 por un 6.
- Si el resto es 3: La voracidad subyacente consiste en alterar el vector numérico minimizando radicalmente la destrucción de la base canónica de cuatros en favor de la combinatoria de mínimos.

■ **Ej. 22: Mínimo de Andenes de Tren (Sweep Line / Línea de barrido)**

Patrón: Variación de Intervalos / Línea de Barrido Cronológica.

Coste: $\mathcal{O}(N \log N)$ (Fruto de la ordenación temporal de los nodos). $\mathcal{O}(N)$ si estuviesen preordenados.

Desarrollo de la Decisión Voraz: Evadimos la farragosa tarea de encajar polígonos de tiempo disociando analíticamente cada servicio en dos partículas atómicas independientes: (*Marca de Hora*, "Llegada") y (*Marca de Hora*, "Salida"). La maniobra voraz clave consiste en **interconectar y ordenar todos estos eventos en una única secuencia cronológica global** (aplicando la salvaguarda de que ante un empate de hora, un evento de salida procesa invariablemente antes que uno de llegada para liberar el andén). Desplegamos un contador $C = 0$ (estado actual) y un memorizador $M = 0$ (máximo histórico). Barremos la línea temporal: una llegada induce una retención de andén ($+1$ a C), mientras que una salida emancipa el recurso (-1 a C). Tras cada iteración registramos $M = \max(M, C)$. La magnitud residual alojada en M constituye la cota óptima de andenes requerida.

Ej. 9: Mochila 0-1 con condición especial (Pesos $\uparrow$ = Beneficios $\downarrow$)

Patrón: Variante de Mochila 0-1 resoluble por Voraz.

Coste: $\mathcal{O}(N \log N)$ (o $\mathcal{O}(N)$ si la lista ya se entrega ordenada).

Desarrollo de la Decisión Voraz: Bajo condiciones normales, la Mochila 0-1 exige Programación Dinámica. Sin embargo, el enunciado impone una restricción mágica: $w_1 \leq w_2 \leq \dots \leq w_n$ y simultáneamente $b_1 \geq b_2 \geq \dots \geq b_n$. Esto significa que el objeto 1 es mejor que el 2 en **todos** los aspectos (pesa menos y vale más), el 2 es mejor que el 3, etc.

La **decisión voraz** es: Ordenar los objetos bajo esta premisa y recorrer la lista introduciendo objetos completos. **En el momento en que un objeto i no quepa en la mochila, el algoritmo termina inmediatamente y devuelve la solución.** ¿Por qué nos detenemos sin mirar el resto? Porque si el objeto i no cabe, implica que $w_i > W_{\text{restante}}$. Como la lista está ordenada por pesos crecientes, cualquier objeto posterior $j > i$ tendrá un peso $w_j \geq w_i$, por lo que es matemáticamente imposible que quepa.

Justificación de la Corrección (Argumento de Intercambio):

1. Sea V la solución voraz que selecciona los primeros k objetos $(1, 2, \dots, k)$. Supongamos por reducción al absurdo que existe una solución óptima O estrictamente mejor que V .
2. Si $O \neq V$, debe existir algún objeto x que nuestro algoritmo voraz cogió ($x \leq k$) pero que O descartó. Para que O no haya cogido x , tuvo que haber llenado ese espacio con otro objeto y que está más adelante en la lista ($y > k$). Por tanto, $y > x$.
3. Por la condición del problema, al ser $y > x$, sabemos con certeza absoluta que el peso de y es mayor o igual ($w_y \geq w_x$) y su beneficio es menor o igual ($b_y \leq b_x$).
4. Si en la solución óptima O **intercambiamos** el objeto y por nuestro objeto x , creamos una nueva solución O' . El peso de la mochila disminuirá o se quedará igual (factibilidad garantizada) y el beneficio total aumentará o se quedará igual.
5. Repitiendo este intercambio para cada diferencia, transformamos O en V sin empeorar jamás el beneficio. Contradicción: O no era estrictamente mejor. **Nuestra solución voraz V es óptima.**